almost the opposite of a race condition: rather than two threads racing to be first, each one is waiting for the other, so neither makes any progress.

3.2.4. Deadlock: the problem and a solution

Imagine that you have a toy that comes in two parts, and you need both parts to play with it—a toy drum and drumstick, for example. Now imagine that you have two small children, both of whom like playing with it. If one of them gets both the drum and the drumstick, that child can merrily play the drum until tiring of it. If the other child wants to play, they have to wait, however sad that makes them. Now imagine that the drum and the drumstick are buried (separately) in the toy box, and your children both decide to play with them at the same time, so they go rummaging in the toy box. One finds the drum and the other finds the drumstick. Now they're stuck; unless one decides to be nice and let the other play, each will hold onto whatever they have and demand that the other give them the other piece, so neither gets to play.

Now imagine that you have not children arguing over toys but threads arguing over locks on mutexes: each of a pair of threads needs to lock both of a pair of mutexes to perform some operation, and each thread has one mutex and is waiting for the other. Neither thread can proceed, because each is waiting for the other to release its mutex. This scenario is called *deadlock*, and it's the biggest problem with having to lock two or more mutexes in order to perform an operation.

The common advice for avoiding deadlock is to always lock the two mutexes in the same order: if you always lock mutex A before mutex B, then you'll never deadlock. Sometimes this is straightforward, because the mutexes are serving different purposes, but other times it's not so simple, such as when the mutexes are each protecting a separate instance of the same class. Consider, for example, an operation that exchanges data between two instances of the same class; in order to ensure that the data is exchanged correctly, without being affected by concurrent modifications, the mutexes on both instances must be locked. However, if a fixed order is chosen (for example, the mutex for the instance supplied as the first parameter, then the mutex for the instance supplied as the second